Guia rápido · Módulo 09: Autenticação, JWT e MVC (boilerplate)

Resumo para consulta rápida. A explicação completa está nos slides e nos arquivos desta pasta.

Camadas do boilerplate

```
src/
├── app.js            configura o Express e as rotas
├── server.js         conecta no banco e sobe o servidor
├── config/database.js
├── routes/           caminho + middlewares + controller
├── middlewares/      autenticar, validarCampos, erro
├── controllers/      lê req, chama o service, responde
├── services/         regras de negócio
├── repositories/     conversa com o banco
├── models/           schemas do Mongoose
└── utils/criarErro.js
```

Regra de ouro: controller não fala com o banco; service não conhece `req` / `res`.

Rotas

Rota	Protegida?
`POST /api/auth/cadastro`	não
`POST /api/auth/login`	não
`GET / PATCH / DELETE /api/usuarios/perfil`	sim (token)

bcrypt

```
const senhaHash = await bcrypt.hash(senha, 10)        // cadastro
const ok = await bcrypt.compare(senha, usuario.senhaHash) // login
```

No model: `senhaHash: { type: String, select: false }`. Para o login: `.select("+senhaHash")`. **Nunca** devolva a senha nem o hash.

JWT

```
const token = jwt.sign({ id: usuario._id, email: usuario.email },
  process.env.JWT_SECRET, { expiresIn: "1d" })
```

Header de toda rota protegida: `Authorization: Bearer <token>`. O JWT é **assinado**, não criptografado: qualquer um lê o conteúdo. Nada sensível no payload.

Middleware autenticar

```
const authHeader = req.headers.authorization
if (!authHeader) return next(criarErro("Token não informado.", 401))
const [tipo, token] = authHeader.split(" ")
if (tipo !== "Bearer" || !token) return next(criarErro("Formato do token inválido.", 401))
const dados = jwt.verify(token, process.env.JWT_SECRET) // lança erro se inválido/expirado
req.usuario = { id: dados.id, email: dados.email }
next()
```

Erros num lugar só

```
throw criarErro("Email já cadastrado.", 409)   // no service
// middleware de erro: 4 parâmetros, registrado por último no app.js
function erroMiddleware(error, req, res, next) {
  return res.status(error.status || 500).json({ message: error.message })
}
```

Autorização por papel

```
papel: { type: String, enum: ["usuario", "admin"], default: "usuario" }
router.get("/", autenticar, autorizar("admin"), UsuarioController.listar)
```

- `autenticar` **antes** de `autorizar`.
- O papel vem do banco/token, **nunca** do body.
- O token guarda o papel antigo até expirar.

Trocar a senha

`PATCH /api/usuarios/senha` com a senha atual e a nova. Senha atual errada → **400** (o usuário está logado; não é 401). O token antigo continua valendo até expirar (`iat`).

Recurso com dono

```
usuario: { type: mongoose.Schema.Types.ObjectId, ref: "Usuario", required: true }
// no controller/service:
const livro = await LivroRepository.criar({ ...dados, usuario: req.usuario.id })
const meus = await LivroRepository.listarPorUsuario(req.usuario.id)
```

Status que importam

400 dado inválido · **401** sem login ou token inválido · **403** logado, mas sem permissão · **404** não existe · **409** conflito (e-mail repetido)

Armadilhas

- Dono vindo do body: qualquer um cria recurso em nome de outro.
- Esqueceu `select: false`: o hash vaza na resposta.
- `JWT_SECRET` fraco ou no GitHub: qualquer um gera tokens válidos.

LionsDev • Professor Nicolas Cardoso Motta

Guia rápido · Módulo 09